

RADAR APP STRATEGY & BLUEPRINT

Hyper-Local Spontaneous Event Discovery Platform

Product Architecture & Market Requirements Document

1. Executive Summary & Market Analysis

The local event discovery landscape is mature yet deeply fragmented. While major tech legacy platforms command vast database volume, consumers experience high friction when seeking real-time, spontaneous physical experiences. This document establishes the strategic, backend, and frontend blueprint for **Radar** (working title), an offline-first, map-centric mobile utility designed to answer the immediate consumer friction: *"What is happening within a 15-minute walk right now?"*

Competitive Landscape Matrix

Competitor Tier	Core Strength	Architectural Gap (Radar Opportunity)
Mass Market Engines (Eventbrite, Ticketmaster)	Unmatched global event volume & transactional infrastructure.	High information noise; filled with online webinars, corporate seminars, and pre-scheduled distant bookings. Poor real-time mapping UI.
Interest Communities (Meetup, Resident Advisor)	High audience retention inside niche hobbies or music subcultures.	Data siloing. Requires consumers to maintain multiple applications for varied weekend interests.
Tourism Aggregators (TripAdvisor, Yelp)	Strong directory of static brick-and-mortar storefronts.	Fails to track ephemeral, transient events like food trucks, pop-up markets, or street performances.

2. Unique Selling Proposition (USP)

Core Architecture Philosophy

"The zero-search map that shows you exactly what's happening within a 15-minute walk, starting right now."

Radar shifts the paradigm from traditional directory lookup to real-time execution. The product identity relies strictly on three absolute engineering rules:

- **The Now-and-Next Constraint:** The primary view hides any entry not occurring actively or starting within a 3-hour window. This algorithmically eliminates 90% of traditional marketplace clutter.
- **The Geofenced Bubble:** Strict geographic bounding box tied to the user's real-time hardware GPS location.
- **Vibe-First Indexing:** Ditching long form text inputs for lightweight, visual map pins indicating temporal urgency, cost, and atmospheric category.

3. The Onboarding Strategy (Zero-Friction Funnel)

Traditional multi-page registration profiles act as user conversion bottlenecks. For a real-time localization utility, the Time-To-Value (TTV) must remain under five seconds.

- 1 **Hardware Permission Gate (Seconds 0-3):** Immediate system location request paired with clear explanatory placeholder micro-copy. Notification registration is completely bypassed during initial session initialization.
- 2 **The Discovery Phase (Seconds 4-10):** The core map interface instantly renders local neighborhood data. No instructional splash overlays or feature cards are used.
- 3 **The Lazy Wall (Seconds 11-30):** The interface operates fully anonymously until high-intent mutations occur (e.g., event bookmarking, booking execution, hosting registration). At this junction, single-tap OAuth protocols (Apple/Google ID) are forced.

4. Data Seeding Pipeline Architecture

To systematically address the cold-start problem, a hybrid three-tier automated synchronization mechanism is used to populate database arrays on Day 1.

The Three-Tier Data Pipeline

1. **Tier 1: Enterprise Event APIs:** Algorithmic ingest of predictable geographical events utilizing platforms like PredictHQ, Eventbrite Public API, and Google Places API to secure structural foundation coordinates.

2. **Tier 2: Automated Web Scraping:** Scheduled low-code scraping routines (via Browse AI or Firecrawl) targeting regional alternative news platforms, artisanal brewery calendars, and municipal tourism boards.
3. **Tier 3: Hand-Curated Seed Layer:** Focused editorial ingestion of highly visual, social-first transient occurrences gathered from micro-influencer geotags and visual community platform trends.

Database Schema & Logic Implementation

The backend requires self-cleaning capabilities to maintain historical irrelevance rules. Data transformation layers (built via Make.com passing data into Airtable or Baserow) calculate coordinates and apply temporal status values.

For calculating temporal relevance natively within the storage architecture, a real-time evaluated boolean status flag is configured:

```
IF(IS_SAME({Event_Start_Date}, TODAY(), 'day'), "Yes", "No")
```

5. Frontend Map Rendering & Custom Visual Logic

Within low-code native cross-platform application frameworks (such as FlutterFlow), the visual design replaces traditional standard marker pins with explicit custom UI layout components mapped to state variables.

Dynamic Component Logic Flow

- **Geographic Bounding Calculation:** Utilizes device GPS matrices dynamically mapped to visual components using a custom client-side distance formula module, establishing strict 1.0-mile display filters.
- **Categorical Color Mutation:** The background fills of the component widgets dynamically reference database strings to parse category tags into explicit color values (e.g., `Category == 'Music' ? Gradient_Red : Gradient_Green`).
- **Urgency Visual Triggers:** A recurring client-side background timer continuously evaluates physical distance and temporal parameters. If an entry reaches an end-time threshold under 30 minutes, conditional rendering rules execute an active pulsing border animation widget, prompting user engagement through perceived systemic urgency.